



Freshman Tutorial

Introduction

우리가 하는 일은 연구(research)이다. 그리고 연구를 뒷받침 하는 건 엔지니어링 스킬이다.

좋은 엔지니어링 스킬이 없이는 아무리 좋은 아이디어가 있어도 실현이 안된다.

그렇기 때문에 대학원 생활을 **정상적으로** 마치게 되면 자신의 연구에 대한 결과와 엔지니어링 스킬을 갖추게 될 것이다.

따라서 본 페이지에서는 우리 연구실의 새내기가 갖추어야할 기본적인 엔지니어링 소양을 소개하고, 기본적인 튜토리얼을 제공할 것이다.

참고로, 본 페이지에서는 키워드만 제공하니 이를 바탕으로 검색하여 원하는 바를 찾도록 하자.

1st step: Install VmWare and Linux

VmWare를 설치해서 Linux 배포판을 설치하자.

주로 사용하는 Linux 배포판은 Ubuntu나 Centos가 있다.

2) 최신 Linux kernel을 다운로드 받자. (2025.9.24. 기준 latest stable kernel version: v6.16.8)

2nd step: Build Kernel

최신 Linux kernel을 빌드하고 설치하자.

깔려있는 리눅스 배포판의 커널 버전을 최신 stable 커널로 바꿔보자.

커널은 여기서 받거나 tarball을 이용하도록 하자.

<http://kernel.org>

3rd step: Modify Kernel

간단한 시스템 콜을 추가해보자.

[Assignment 2] Implementing a new system call getmemutil().

참고자료

- Linux Kernel Development 3rd → System call chapter
- Google

커널 버전이 많이 바뀌었기 때문에, 책만 보고 커널에 시스템 콜을 추가하는 것은 어려울 것이다.

검색해보자. 구글은 나보다 많이 알고 있다.

그리고 여기까지가 학부 수준에서 해봤을 법한 단계이다.

4th step: Build Benchmark

벤치마크를 하나 선정해서 빌드해보자.

이 단계에서 사용해볼만한 벤치마크는 다음과 같다.

- Filebench
- **fio (GitHub - axboe/fio: Flexible I/O Tester)**
 - sequential write/read (2G file, sync&async)
 - random write/read (2G file, sync&async)
 - sequential write 후 random read (2G file 대상으로 4k read)

뭘 써야할지 잘 모르겠거든, 사수한테 물어보자.

벤치마크 프로그램은 보통 빌드 및 실행을 위해 요구하는 라이브러리 및 유틸리티들이 있다.

따라서 본 단계에서 요구하는 바는 다음과 같다.

- Make 및 Configure의 사용법을 알고 있는가?
- 필요한 라이브러리 및 유틸리티들을 설치할 수 있는가?

5th step: Do Experiment

실험 결과를 뽑아보자.

실험 결과는 각 벤치마크의 종류에 따라 다른 형태가 될 수 있으며, 벤치마크의 리포트 뿐 아니라 시스템 유틸라이제이션을 체크하기 위해 추가로 모니터링 용 스크립트를 작성해야 할 것이다.

따라서 본 단계에서 요구하는 바는 다음과 같다.

- 벤치마크 프로그램의 올바른 실행 (조건을 설정할 수 있어야 함)
- 원하는 리소스의 사용량을 출력할 수 있는 시스템 인터페이스를 알고 있는가?

- 원하는 리소스의 사용량을 모니터링할 수 있는 셸 스크립트를 작성할 수 있는가?

6th step: Results Refinement

실험결과를 뽑아내었지만, 그 자체로는 아무런 의미도 없다. 가공되지 않은 데이터는 그냥 데이터일 뿐이다.

원하는 정보를 뽑아낼 수 있도록 데이터를 가공하라.

데이터를 가공하는 방법은 여러 가지가 있다. 널리 쓰이는 방법을 소개하자면 다음과 같다.

- Python은 간단하면서 강력한 언어이다. 특히 문자열 처리를 잘 지원하기 때문에, 파이썬을 활용한다면 원하는 형태로 데이터를 가공할 수 있을 것이며, 이를 바탕으로 통계치를 계산할 때도 아주 유용하다.
- AWK는 셸에서 사용할 수 있는 유용한 언어이다. 간단한 데이터 가공은 AWK만으로도 가능하며, 심지어 계산 기능도 제공한다.

7th step: Plotting

데이터를 잘 가공했다면, 이제는 도표로 만들 차례이다. 논문에 숫자 그대로의 데이터가 실리는 경우는 거의 없으며, 대부분 그래프의 형태로 시각화하여 표현한다.

도표를 만들 수 있는 방법은 다음과 같다.

- 데이터의 양이 적을 경우, 엑셀을 활용할 수 있다.
- 하지만 대부분의 경우, gnuplot을 사용하여 도표를 작성한다. 이를 통해 만들어지는 그래프는 크기에 구애받지 않으며, 늘리면 늘리는대로 스케일링 된다. 그래프 자체가 스크립트이기 때문이다.
- 최근에는 gnuplot보다는 pyplot을 더 많이 사용한다. Python을 통해 작성되기 때문에 진입 장벽도 낮은 편이니 pyplot으로 작성하는 편을 추천한다.

8th step: Version Management

연구는 혼자 수행하기도 하지만 공동으로 수행하기도 한다.

그리고 둘 모두의 경우 소스 코드의 버전관리는 매우 중요한 문제가 된다.

한달간 열심히 코드를 수정했는데 덮어쓰우기로 인해 코드를 날려봤는가? 한번 겪어보면 모든 일을 시작하기 전에 git부터 사용할 것이다.

대부분의 경우 git으로 소스 코드 관리를 한다.

Git 사용법의 경우 구글에 물어보자.

9th step: System Programming

시스템 프로그래밍 입문이다.

"**/dev/csl**"로 표시되는 block device를 만들기 위한 모듈 코드를 작성해보자.

모듈이란 모놀리식 커널의 단점을 보완하기 위해 만들어진 확장 가능하며 커널에 넣었다 뺐다 할 수 있는 기능이라고 생각하면 된다.

다음을 참고하라.

[Linux Device Driver] Chapter 16: Block Drivers

System Programming Study

위 모듈을 load/remove 할 때, 적절한 메시지를 dmesg에 띄우도록 하자

10th step: Documentation

문서화다.

여태했던 작업들을 한번 위키나 자신이 사용하는 도큐멘테이션 툴로 문서화 해보자.

문서화는 꽤 중요한 작업이다. 문서화안하고 일을 진행하다가는 나중에 자신의 기억력을 탓하게 될 날이 **반드시** 온다.

step 별로 문서화해서 **사수**에게 컨펌받자.

Finale

축하한다.

당신은 이 코스를 모두 끝마쳤다.

이것으로써 당신은 어디가서 이렇게 말할 수 있다.

"저 대학원 입학했어요."

fin.